

Lab 3 - FHIR Servers (OMOPonFHIR)

- Due Feb 16 by 8am
- Points 5
- Submitting an external tool
- Available Jan 12 at 8am - Feb 16 at 8am

Lab 3 - FHIR Servers (OMOPonFHIR)

Overview

The Observational Medical Outcomes Partnership Common Data Model (OMOP CDM) is an important schema used in healthcare research. Traditionally, moving data between from clinical systems into research databases has been an often labor intensive manual ETL (Extract, Transform, Load) process. This makes it a prime example of where data modernization in the healthcare space can help improve the sharing of data among a variety of systems, and even provide simple translation into other data formats or standards.

This lab takes a look at a Georgia Tech and Georgia Tech Research Institute project: OMOPonFHIR. OMOPonFHIR is one of the earliest examples of leveraging FHIR to write clinical data into an OMOP CDM database, and is in active use by a number of major stakeholders in healthcare research such as other universities and commercial entities. It is a fully open source Java implementation, that layers a HAPI FHIR server on top of an OMOP CDM database, with mapping in the middle. The key part of OMOPonFHIR is that mapping layer, as the OMOP CDM and FHIR are vastly different standards. As such, mapping between them requires a deep understanding of both data structures in terms of both technical structure and semantic meaning.

As a brief introduction to the complexities of working across standards such as this, and how FHIR is being leveraged by stakeholders to solve problems long endemic to healthcare IT, this lab will have you deploy an OMOPonFHIR server using Docker, perform some simple exercises, and provide some broad reflection on the process.

Learning Objectives

- Overview of the OMOP Common Data Model.
- Implementing a strategy that supports OMOPonFHIR data mappings.
- Deploying OMOPonFHIR.

Tools Used

- Lab Deployment Repository - <https://github.gatech.edu/cs6440-students/Lab-3-OMOP-on-FHIR>  (<https://github.gatech.edu/cs6440-students/Lab-3-OMOP-on-FHIR>)
- Poetry (Python Package Manager) or Pip and some Environment Manager
 - **(READ INSTRUCTIONS CLOSELY!)**
- **Docker**  (<https://docs.docker.com/engine/install/>)
- **Pgadmin4**  (<https://www.pgadmin.org/download/>), DataGrip, or similar for exploring the OMOP CDM database
- **OHDSI Athena**  (<https://athena.ohdsi.org>)

Additional Resources

- **OMOP Common Data Model version 5.4**  (<https://ohdsi.github.io/CommonDataModel/cdm54.html>)
- **OMOPonFHIR Database Extensions**  (<https://github.com/SmartChartSuite/OMOP-5.4-PostgreSQL/tree/main/omoponfhir>)
- Clinical Predictive Modeling Development and Deployment with OMOP CDM and FHIR - https://www.ohdsi.org/web/wiki/lib/exe/fetch.php?media=resources:gt_ohdsi-abs-final.pdf  (https://www.ohdsi.org/web/wiki/lib/exe/fetch.php?media=resources:gt_ohdsi-abs-final.pdf)
 - This is the original publication on what became OMOPonFHIR from 2015.

Getting Started

Deploying OMOPonFHIR

For this lab, a simplified deployment using Docker Compose has been provided. Clone or download the required files from the following repo: <https://github.gatech.edu/cs6440-students/Lab-3-OMOP-on-FHIR>  (<https://github.gatech.edu/cs6440-students/Lab-3-OMOP-on-FHIR>)

 (<https://github.gatech.edu/cs6440-hub/Lab-6-OMOPonFHIR>) Before running the Docker Compose file, you must first do some configuration and setup. To function properly, OMOP Databases require loading vocabulary files which populate the database with concepts from major code systems (e.g. LOINC, SNOMED CT, etc.). Here we will be using a bare minimum set of vocabulary for the sake of speed, as a complete vocabulary can take many hours to load and is error prone with memory concerns in a local Docker environment.

To download the vocabulary files, go to <https://athena.ohdsi.org>  (<https://athena.ohdsi.org/>). You will need to accept the license agreement and register an account. Click the “Login” button in the top right corner and follow the instructions. After you are logged in, click the download button in the top bar to begin the download process.



Select the following Vocabularies from the list of available options:

- SNOMED
- LOINC
- RxNorm
- Gender (OMOP Gender)
- Race (Race and Ethnicity Code Set (USBC))

Failure to download any of these vocabularies may cause issues with your deployment or errors in mapping, and require clearing your database and starting over.

Once you have the above selected, select the “Download Vocabularies” button. In most cases, your vocabulary package will be available for download in a few minutes. Sometimes, under heavy load, it may take a bit longer. In any case, you will receive an email when your vocabulary package is available.

Navigate to your lab folder that you cloned/downloaded previously. Extract the .csv files into the “vocab” folder (a placeholder file is inside the folder to ensure you are clear precisely where to place the CSVs).

Open a terminal and navigate into the lab folder here. From the root directory, deploy your Docker instance by running:

```
docker compose up
```

A Docker compose file essentially acts as a way to network multiple docker containers together. Here you are deploying both OMOPonFHIR and a Postgres database for the OMOP CDM. You may wish to open the docker-compose.yaml file and look over exactly how it is setup for familiarity. While typically it is required to manually configure the OMOP CDM schema, the Docker image provided will automatically load the SQL scripts necessary as well as copy in the vocabulary. This particular Docker Compose approach also includes a simple health check to account for when the database is ready to go before starting up OMOPonFHIR. (If it begins without the expected tables found, it will crash.)

Note: On average, the process will take around 10 minutes to complete. During the build, you will see an error message that reads – ERROR: relation “f_person” does not exist at character 15. Disregard this, this is the byproduct of the health check.

Once the process is complete, you can access the OMOPonFHIR UI by launching the following URL within your browser: <http://localhost:8080/> ➞ [\(http://localhost:8080/\)](http://localhost:8080/). Note that it may not yet be evident if there is a hidden error with the deployment until you test it.

To test OMOPonFHIR, look in the `/resources/` folder and you will find a number of FHIR JSON files. In the UI or through a tool such as Postman, post `patient_1.json` to the server. To do this through the UI, select "Patient" in the resource list on the left, then go to "CRUD Operations". Scroll down to "Create" and copy and paste the file's content into the text area provided, then press the create button.

Setting Up Starter Code

This lab is compatible with Python ≥ 3.10 . There are significant changes made in Python 3.10, so if you have an older version installed you will need to upgrade. (Note for Windows users: The official Python installer does not add itself to the Windows path entirely, you may wish to install through the Windows Store instead.)

For this lab, we have traditionally encouraged the use of [Python Poetry \(https://python-poetry.org/docs/\)](https://python-poetry.org/docs/) for which it was originally built, which is a tool for managing Python packages and environments. It saw increased popularity for a while, with many believing it to more modern. However, we have noticed some issues overtime and no longer endorse a specific approach. Instructions for Poetry are given below, but if you are more familiar with Conda or Pip, or even just working with VS Code environments, you should use what you are familiar with, there are not many dependencies to switch over.

Poetry Instructions

First, go to the Poetry website and follow the installation according to your operating system: <https://python-poetry.org/docs/#installing-with-the-official-installer> (<https://python-poetry.org/docs/#installing-with-the-official-installer>)

After installation, navigate into the `/student/` folder with the `pyproject.toml` file. You can spawn a virtual environment by running:

```
poetry install # Creates environment
poetry shell   # Enters environment
```

To execute your code, you will use python to access the `main.py` file.

```
python3 main.py    // Note: Replace python3 with whatever your local environment uses as a
link.
```

You may also test your code using `pytest`. Three simple tests are given for the parts you need to implement actual code. These tests should provide you with a strong indication your code is working. (Gradescope has additional, hidden tests, but none are designed to test edge cases or attempt to trip you up.) To run `pytest`, execute it from within the poetry shell as below

```
pytest
```

-- OR --

```
poetry run pytest
```

Dependencies

Please note that you cannot (and should not need to) add additional dependencies to the project or it will not work on Gradescope. All code is expected to be very simple, and you are given everything you need.

Connecting to your Database

For browsing your database, you will need a tool such as [PG Admin 4](https://www.pgadmin.org/download/)  (<https://www.pgadmin.org/download/>) or [DataGrip](https://www.jetbrains.com/datagrip/)  (<https://www.jetbrains.com/datagrip/>). (Note: As with IntelliJ, as a GT student you have access to a free license for DataGrip.) You may also manually query the tables with SQL, but using one of the graphical interfaces is generally far more efficient for this lab. Once you have installed your application of choice, configure a connection to your running Docker container database.

Configuration Settings

- **Name:** Omop54 Postgres (Or anything else you'd like)
- **Host:** localhost
- **Port:** 5432 (Change as needed if you had to remap the port)
- **User:** postgres
- **Password:** password
- **Database:** omop54
- **URL:** jdbc:postgresql://localhost:5432/omop54

Note: The database password is configured in the Docker Compose file, the default is just "password".

The following screenshot shows a basic configuration in Datagrip. Please be aware this is provided as a sample only, and the values may not align with this lab. If you want to however, you may opt to connect to the default database of "postgres", though you will need to select to also show the "omop54" database. To see other databases in DataGrip, click to the right in the tree where it will say something such as "1 of 3", which indicates how many of the databases on the server are

currently being shown.

The screenshot shows the 'Data Sources and Drivers' dialog box. The left sidebar has tabs for 'Data Sources', 'Drivers', and 'DDL Mapping'. Under 'Data Sources', there are icons for adding, removing, and refreshing sources. A list of 'Project Data Sources' shows 'postgres@localhost' selected. Below this is a 'Problems' section. The main area on the right is for configuring the selected data source. It has tabs for 'General', 'Options', 'SSH/SSL', 'Schemas', and 'Advanced'. The 'General' tab is active, showing fields for Name, Comment, Connection type, Driver, Host, Port, Authentication, User, Password, Save, Database, and URL. The 'Test Connection' button is at the bottom left of the configuration area. At the very bottom of the dialog are 'OK', 'Cancel', and 'Apply' buttons.

Questions and Exercises

For your deliverable for this assignment, the work you submit to Gradescope will all be in the `student_code.py` file. However, you will have to work with the `main.py` as well as you develop your code to test it.

Each question and exercise is documented fully inside the `student_code.py` file, but will be summarized here as well.

Questions

These are simple questions that ask you to post the FHIR resources provided to the FHIR server, and then understand how they are written to the database or how certain accommodations are

made in what is returned to users on a request for a given resource.

1. After posting patient_1.json to the OMOPonFHIR it will be written to the OMOP database. If you then read the patient back from the FHIR server, additional data may be added as part of standardized representation. For example, the U.S. Core Race element is not present on in the patient_1.json file, but it will be generated when the resource is requested. What is the *code* returned for the US Core Race's coding in this case? **(Case Sensitive!)**
2. Three Observation resources are given, observation_1.json through observation_3.json. FHIR Observations line up with two different tables based on how it is categorized in the concept tables from the loaded vocabulary. Post all three to the OMOPonFHIR Server. Which of the two OMOP CDM tables does each get stored in? **(Case Sensitive! Use all lower case.) NOTE: These are separated out into their own sub questions in the student_code.py file.**
3. Now, post observation_4.json to the FHIR Server. This file gets written to the observation table. Due to complexities in mapping, the valueCodeableConcept field in the original FHIR resource is replaced with what field when requested back from the server? **(Answer with the full string for the field key, exactly as it is given. Case Sensitive!)**
4. Moving to the two MedicationStatement resources, post both to the FHIR Server if you have not. What OMOP CDM table are they populated into? **(Case Sensitive! Use all lowercase.)**

Exercises

There are three exercises with require minor code implementations. As this is not intended to be a SQL focused lab, we have set this up using SQLAlchemy along with some simple SQL examples in main.py for your database connection. All database connections are built for you by the provided utility files. You are only required to consider what information you need and return it appropriately. For these, please reference the student_code.py file which includes example inputs and outputs for each exercise. You can see this through the test_simple.py file under the tests folder as well (this is the file that is executed when you run pytest).

Submission

Your student_code.py file should be submitted to Gradescope under Lab3. No other files need to be submitted. Please see the Lab 2 Gradescope submission information for more details on Gradescope.

Submit Programming Assignment

 Upload all files for your submission

Submission Method

 Upload

 GitHub

 Bitbucket

Drag & Drop

Any file(s) including .zip. Click to browse.